An Introduction to Reinforcement Learning

CS115B – Spring 2026

James Pustejovsky

Why an RL lecture in CS115

- RL is one of the main paradigms for learning from **interaction** rather than static labels.
- It gives students the vocabulary behind **policies, rewards, trajectories, exploration, credit assignment**, and **planning**.
- That vocabulary later appears in modern NLP and LLM post-training: RLHF, PPO, reward models, bandits, online learning, and preference optimization.
- Today we focus on **general RL**, not just LLMs.

Big picture

Supervised learning asks "what is the right output?"; RL asks "what sequence of actions leads to high long-term return?"

Where RL fits among learning paradigms

Supervised

- Input-output pairs
- Immediate labels
- Independent examples

Unsupervised / self-supervised

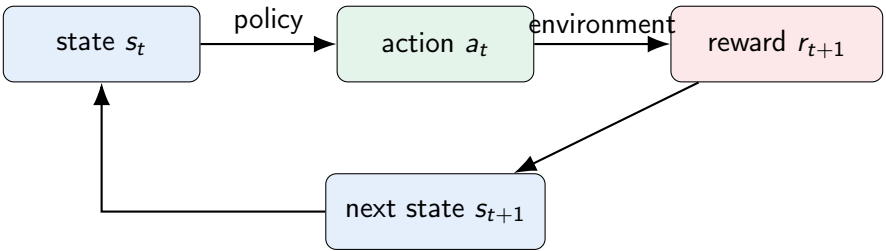
- Structure in raw data
- No explicit reward
- Representation learning

Reinforcement learning

- Agent acts in an environment
- Receives rewards
- Must optimize long-term outcomes

Delayed consequences are the core difficulty in RL.

The interaction loop



A trajectory is a sequence

$$(s_0, a_0, r_1, s_1, a_1, r_2, \dots).$$

The learner wants to choose actions so that accumulated reward is large.

Reward, return, and the discount factor

- **Reward** r_{t+1}: immediate scalar feedback.
- **Return** G_t: total future reward from time t onward.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$$

- $\gamma \in [0, 1)$ is the **discount factor**.
- Smaller γ: more myopic behavior.
- Larger γ: more emphasis on long-term consequences.

Interpretation

Discounting can encode uncertainty, finite attention horizon, or simply make the infinite sum well behaved.

Markov decision processes (MDPs)

An RL task is often modeled as an MDP with:

- states $\mathcal{S}$
- actions $\mathcal{A}$
- transition dynamics $P(s' \mid s, a)$
- reward function $R(s, a, s')$
- discount factor γ

Markov property

The next state depends on the current state and action, not on the full earlier history.

If the agent cannot observe the full state, we move toward **partial observability** (POMDPs), where memory becomes important.

Policies: how the agent acts

- A **policy** tells the agent what action to take.
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a \mid s)$ gives a distribution over actions.

Why stochastic policies?

- exploration
- robustness
- natural fit for policy gradients

In NLP/LLMs

A language model already defines a token distribution, so stochastic policies feel natural.

State-value and action-value functions

For a policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s]$$

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a]$$

- $V^{\pi}(s)$: how good it is to be in state s.
- $Q^{\pi}(s, a)$: how good it is to take action a in state s.
- Optimal values define the best achievable long-term return.

$$V^{*}(s) = \max_{\pi} V^{\pi}(s), \qquad Q^{*}(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

Bellman equations

Bellman expectation equation

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s',r} P(s', r \mid s, a)\big[r + \gamma V^{\pi}(s')\big]$$

Bellman optimality equation

$$V^{*}(s) = \max_{a} \sum_{s',r} P(s', r \mid s, a)\big[r + \gamma V^{*}(s')\big]$$

- These equations express a recursive decomposition of long-term value.
- Nearly every major RL algorithm is a way of exploiting this recursion.

The exploration-exploitation dilemma

Exploit

Choose the action that currently looks best.

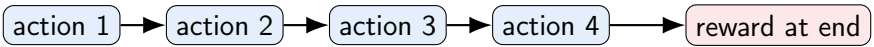
Explore

Try actions that may reveal better long-term strategies.

- Too little exploration $\rightarrow$ stuck with mediocre behavior.
- Too much exploration $\rightarrow$ poor short-term performance.
- This issue already appears in the simpler **multi-armed bandit** setting.

Credit assignment and sparse rewards

- A reward may arrive long after the actions that caused it.
- Many tasks have **sparse rewards**: feedback appears only occasionally.
- Example: in a long dialogue, which earlier turn caused the final success or failure?



Value functions and return estimates help propagate delayed reward backward through time.

Two high-level families of methods

Value-based methods

Learn values such as $Q(s, a)$ and act by choosing high-value actions.

Policy-based methods

Optimize the policy directly, often with gradients.

Actor-critic methods

Combine both: an **actor** chooses actions and a **critic** estimates value.

This actor-critic vocabulary is especially useful later for PPO and RLHF.

Q-learning in one slide

- Q-learning is an off-policy temporal-difference method.
- It updates action values from sampled transitions.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big]$$

- The bracketed term is the **TD error**.
- If state and action spaces are small, a table can work.
- For large spaces, we need function approximation.

Limitation

For language generation, the action space is huge, which makes pure Q-learning awkward.

Policy gradients in one slide

Instead of learning action values and taking an argmax, we directly optimize a parameterized policy π_θ.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\big[G(\tau)\big]$$

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\right]$$

- This is the core REINFORCE idea.
- Increase probability of actions that led to high return.
- Decrease probability of actions that led to poor return.

Why baselines and advantages help

Plain policy gradient estimates can have high variance.

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big(G_t - b(s_t)\big)\right]$$

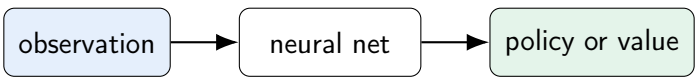
- $b(s_t)$ is a **baseline**, often an estimate of the value function.
- The difference $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$ is the **advantage**.
- Advantage tells us whether an action was better or worse than expected in that state.

Intuition

The critic says what was expected; the actor learns from whether reality was better or worse.

From tabular RL to deep RL

- Tabular methods store one value per state or state-action pair.
- Deep RL uses neural networks to approximate policies or value functions.
- This enables image, text, and continuous-control problems.



But function approximation introduces instability, overestimation, and sensitivity to hyperparameters.

Practical headaches in RL

- **Sample inefficiency**: agents may need huge amounts of experience.
- **Reward misspecification**: the reward may not capture what we really want.
- **Instability**: gradients can be noisy; learning can collapse.
- **Distribution shift**: the agent changes the data it sees as it improves.
- **Evaluation**: average reward may hide safety or robustness failures.

One famous lesson

RL systems optimize the metric you give them, not the intention you had in mind.

Reward design and reward hacking

- If the reward is flawed, the policy may exploit the loophole.
- This is called **reward hacking** or **specification gaming**.

Examples

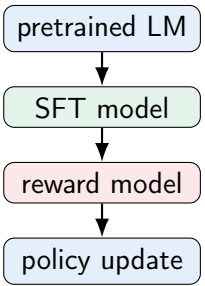
- game agent loops to collect points forever
- recommender learns clickbait
- dialogue model over-optimizes superficial preferences

Mitigations

- better reward design
- constraints and safety checks
- human feedback
- offline evaluation and auditing

From RL to RLHF: a preview

1. Start from a pretrained language model.
2. Do supervised fine-tuning (SFT).
3. Learn a reward model from preference comparisons.
4. Optimize the policy with an RL-style update, often PPO in the classic pipeline.



You do **not** need the LLM details yet; the point is that the basic RL vocabulary transfers directly.

Why PPO became popular in post-training

- PPO is a policy-gradient-style method with clipped updates.
- It tries to improve the policy while avoiding overly large parameter steps.
- In LLM post-training, this helps keep the updated model near a reasonable reference policy.

Pedagogical takeaway

For this class, remember the concepts rather than the exact PPO objective: actor, critic, advantage, trust region intuition, and stable improvement.

Where general RL ideas map onto language modeling

RL term	NLP / LLM analogue
state	prompt, dialogue context, partial generation
action	token, action choice, tool call, response
policy	language model distribution over next steps
reward	preference score, task success, utility, safety metric
trajectory	full generated sequence or multi-turn interaction
critic / value	estimate of future usefulness or return
exploration	sampling, candidate generation, online experimentation

This analogy is imperfect, but it is a useful mental bridge.

What RL is good at - and not good at

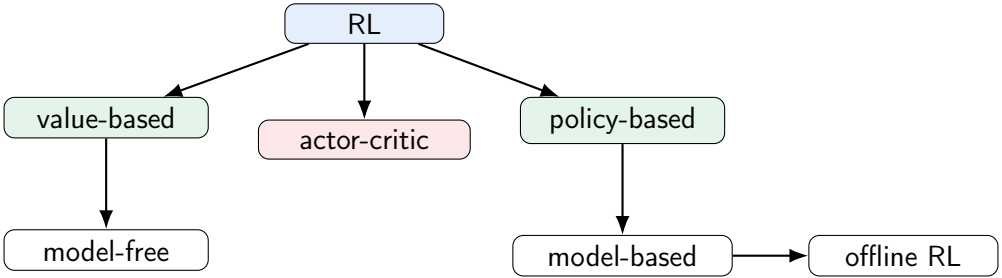
Good fit

- long-term objectives
- sequential choices
- interactive systems
- delayed and partial feedback

Hard cases

- badly specified rewards
- sparse or unsafe exploration
- weak simulators
- very expensive evaluation

A compact taxonomy to remember



Do not memorize every branch. Memorize the organizing questions:

- Are we learning values, policies, or both?
- Are we using a model of the environment?
- Are we learning online or from a fixed dataset?

Summary

- RL studies how agents learn to act to maximize long-term reward.
- The core objects are states, actions, rewards, returns, policies, and value functions.
- Major difficulties are exploration, credit assignment, sparse rewards, and reward misspecification.
- Major families are value-based, policy-based, and actor-critic methods.
- These ideas reappear in NLP through bandits, sequence optimization, RLHF, and preference-based post-training.

One sentence to remember

RL is what you use when the quality of an action depends on what happens later.

References


Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition.


Csaba Szepesvari. *Algorithms for Reinforcement Learning*.


David Silver. *Reinforcement Learning Course* (video lectures).


John Schulman et al. Proximal Policy Optimization Algorithms.


Long Ouyang et al. Training language models to follow instructions with human feedback.